МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ

“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №6

Специальность ПО-13

Выполнил:

Д. А. Марковский

студент группы ПО-13

Проверил:

А. А. Крощенко

Доцент кафедры ИИТ

27.03.2026

Брест 2026

Цель работы:

Освоить приемы тестирования кода на примере использования пакета pytest.

Задание 1:

1. Создайте файл test_cart.py. Реализуйте следующие тесты:
• Проверка добавления товара: после add_item("Apple", 10.0) в корзине должен быть один элемент.
• Проверка выброса ошибки при отрицательной цене.
• Проверка вычисления общей стоимости (total()).
2. Протестируйте метод apply_discount с разными значениями скидки:
• 0% - цена остаётся прежней
• 50% - цена уменьшается вдвое
• 100% - цена становится ноль
• < 0% и > 100% - должно выбрасываться исключение
Используйте @pytest.mark.parametrize
3. Создайте фикстуру empty_cart, которая возвращает пустой экземпляр Cart
@pytest.fixture def empty_cart(): return Cart()
Используйте эту фикстуру в тестах, где нужно создать новую корзину.
4. Допустим, у нас есть функция, которая логирует покупку в удалённую систему:
import requests
def log_purchase(item): requests.post("https://example.com/log", json=item)
• Замокайте requests.post, чтобы не было реального HTTP-запроса
• Убедитесь, что он вызывается с корректными данными
5. Добавьте поддержку купонов:
def apply_coupon(cart, coupon_code):
  coupons = {"SAVE10": 10, "HALF": 50}
   if coupon_code in coupons:
    cart.apply_discount(coupons[coupon_code])
  else:
    raise ValueError("Invalid coupon")
• Напишите тесты на apply_coupon
• Замокайте словарь coupons с помощью monkeypatch или patch.dict

Код программы:

shopping.py:

```
import requests

COUPONS = {"SAVE10": 10, "HALF": 50}


class Cart:
    def __init__(self):
        self.items = []

    def add_item(self, name, price):
        if price < 0:
            raise ValueError("Цена не может быть отрицательной")
        self.items.append({"name": name, "price": price})

    def total(self):
```

```
        return sum(item["price"] for item in self.items)

    def apply_discount(self, discount_percent):
        if discount_percent < 0 or discount_percent > 100:
            raise ValueError("Скидка должна быть от 0 до 100 процентов")

        for item in self.items:
            item["price"] = item["price"] * (100 - discount_percent) / 100


def log_purchase(item):
    response = requests.post("https://example.com/log", json=item, timeout=30)
    response.raise_for_status()
    return response


def apply_coupon(cart, coupon_code):
    if coupon_code in COUPONS:
        cart.apply_discount(COUPONS[coupon_code])
    else:
        raise ValueError("Invalid coupon")
```

test_cart.py

```
# pylint: disable=redefined-outer-name
from unittest.mock import patch, MagicMock
import pytest

import requests

from shopping import Cart, log_purchase, apply_coupon


# 3
@pytest.fixture
def empty_cart():
    return Cart()


@pytest.fixture
def cart_with_items():
    cart = Cart()
    cart.add_item("Apple", 100.0)
    cart.add_item("Banana", 200.0)
    cart.add_item("Orange", 300.0)
    return cart


# 1
def test_add_item(empty_cart):
    empty_cart.add_item("Apple", 10.0)
    assert len(empty_cart.items) == 1
    assert empty_cart.items[0]["name"] == "Apple"
    assert empty_cart.items[0]["price"] == 10.0


def test_negative_price(empty_cart):
    with pytest.raises(ValueError, match="Цена не может быть отрицательной"):
        empty_cart.add_item("Apple", -10.0)


def test_total(empty_cart):
    empty_cart.add_item("Apple", 10.0)
    empty_cart.add_item("Banana", 20.0)
    empty_cart.add_item("Orange", 30.0)
```

```
    assert empty_cart.total() == 60.0


# 2
@pytest.mark.parametrize("discount, expected_prices", [
    (0, [10.0, 20.0, 30.0]),
    (50, [5.0, 10.0, 15.0]),
    (100, [0.0, 0.0, 0.0]),
])
def test_apply_discount_valid(empty_cart, discount, expected_prices):
    empty_cart.add_item("Apple", 10.0)
    empty_cart.add_item("Banana", 20.0)
    empty_cart.add_item("Orange", 30.0)

    empty_cart.apply_discount(discount)

    for i, expected_price in enumerate(expected_prices):
        assert empty_cart.items[i]["price"] == expected_price

    assert empty_cart.total() == sum(expected_prices)


@pytest.mark.parametrize("invalid_discount", [-10, -1, 101, 150])
def test_apply_discount_invalid(empty_cart, invalid_discount):
    empty_cart.add_item("Apple", 10.0)
    with pytest.raises(ValueError, match="Скидка должна быть от 0 до 100
процентов"):
        empty_cart.apply_discount(invalid_discount)


def test_empty_cart_total(empty_cart):
    assert empty_cart.total() == 0.0


def test_multiple_discounts(empty_cart):
    empty_cart.add_item("Apple", 100.0)

    empty_cart.apply_discount(20)
    assert empty_cart.items[0]["price"] == 80.0

    empty_cart.apply_discount(10)
    assert empty_cart.items[0]["price"] == 72.0


def test_add_item_after_discount(empty_cart):
    empty_cart.add_item("Apple", 100.0)
    empty_cart.apply_discount(20)

    empty_cart.add_item("Banana", 50.0)

    assert empty_cart.items[0]["price"] == 80.0
    assert empty_cart.items[1]["price"] == 50.0
    assert empty_cart.total() == 130.0


# 4
@patch('shopping.requests.post')
def test_log_purchase_success(mock_post):
    mock_response = MagicMock()
    mock_response.raise_for_status.return_value = None
    mock_post.return_value = mock_response

    item = {"name": "Apple", "price": 10.0}
    result = log_purchase(item)

    mock_post.assert_called_once_with(
        "https://example.com/log",
```

```
        json=item,
        timeout=30
    )

    mock_response.raise_for_status.assert_called_once()

    assert result == mock_response


@patch('shopping.requests.post')
def test_log_purchase_with_multiple_items(mock_post):
    mock_response = MagicMock()
    mock_response.raise_for_status.return_value = None
    mock_post.return_value = mock_response

    cart = Cart()
    cart.add_item("Apple", 10.0)
    cart.add_item("Banana", 20.0)
    cart.add_item("Orange", 30.0)

    for item in cart.items:
        log_purchase(item)

    assert mock_post.call_count == 3

    expected_calls = [
        (("https://example.com/log",), {"json": {"name": "Apple", "price": 10.0},
"timeout": 30}),
        (("https://example.com/log",), {"json": {"name": "Banana", "price":
20.0}, "timeout": 30}),
        (("https://example.com/log",), {"json": {"name": "Orange", "price":
30.0}, "timeout": 30})
    ]

    for i, call in enumerate(mock_post.call_args_list):
        assert call[0] == expected_calls[i][0]  # args
        assert call[1] == expected_calls[i][1]  # kwargs


@patch('shopping.requests.post')
def test_log_purchase_with_empty_item(mock_post):
    mock_response = MagicMock()
    mock_response.raise_for_status.return_value = None
    mock_post.return_value = mock_response

    item = {}
    result = log_purchase(item)

    mock_post.assert_called_once_with(
        "https://example.com/log",
        json={},
        timeout=30
    )
    assert result == mock_response


@patch('shopping.requests.post')
def test_log_purchase_with_http_error(mock_post):
    mock_response = MagicMock()
    mock_response.raise_for_status.side_effect =
requests.exceptions.HTTPError("404 Not Found")
    mock_post.return_value = mock_response

    item = {"name": "Apple", "price": 10.0}

    with pytest.raises(requests.exceptions.HTTPError, match="404 Not Found"):
        log_purchase(item)
```

```
    mock_post.assert_called_once_with(
        "https://example.com/log",
        json=item,
        timeout=30
    )


@patch('shopping.requests.post')
def test_log_purchase_integration_with_cart(mock_post):
    mock_response = MagicMock()
    mock_response.raise_for_status.return_value = None
    mock_post.return_value = mock_response

    cart = Cart()
    cart.add_item("Apple", 10.0)
    cart.add_item("Banana", 20.0)

    cart.apply_discount(50)

    for item in cart.items:
        log_purchase(item)

    expected_items = [
        {"name": "Apple", "price": 5.0},
        {"name": "Banana", "price": 10.0}
    ]

    for i, call in enumerate(mock_post.call_args_list):
        assert call[1]["json"] == expected_items[i]


# 5
def test_apply_valid_coupon_save10(cart_with_items):
    original_total = cart_with_items.total()
    apply_coupon(cart_with_items, "SAVE10")
    assert cart_with_items.total() == original_total * 0.9


def test_apply_valid_coupon_half(cart_with_items):
    original_total = cart_with_items.total()
    apply_coupon(cart_with_items, "HALF")
    assert cart_with_items.total() == original_total * 0.5


def test_apply_invalid_coupon(cart_with_items):
    with pytest.raises(ValueError, match="Invalid coupon"):
        apply_coupon(cart_with_items, "INVALID")


def test_apply_coupon_to_empty_cart(empty_cart):
    apply_coupon(empty_cart, "SAVE10")
    assert empty_cart.total() == 0.0


def test_apply_coupon_with_mocked_dict_patch(cart_with_items):
    with patch.dict('shopping.COUPONS', {'TEST50': 50}, clear=True):
        apply_coupon(cart_with_items, "TEST50")
        assert cart_with_items.total() == 600 * 0.5


def test_apply_coupon_with_multiple_mocked_coupons():
    test_coupons = {
        'TEST20': 20,
        'TEST30': 30,
        'TEST70': 70
    }
```

```
    with patch.dict('shopping.COUPONS', test_coupons, clear=True):
        for coupon_code, discount in test_coupons.items():
            cart_copy = Cart()
            cart_copy.add_item("Item", 100.0)
            apply_coupon(cart_copy, coupon_code)
            assert cart_copy.total() == 100.0 * (100 - discount) / 100


def test_apply_coupon_with_empty_mocked_dict(cart_with_items):
    with patch.dict('shopping.COUPONS', {}, clear=True):
        with pytest.raises(ValueError, match="Invalid coupon"):
            apply_coupon(cart_with_items, "SAVE10")


def test_apply_coupon_with_monkeypatch(monkeypatch, cart_with_items):
    test_coupons = {'MONKEY20': 20, 'MONKEY80': 80}

    monkeypatch.setattr('shopping.COUPONS', test_coupons)

    original_total = cart_with_items.total()
    apply_coupon(cart_with_items, "MONKEY20")
    assert cart_with_items.total() == original_total * 0.8

    with pytest.raises(ValueError, match="Invalid coupon"):
        apply_coupon(cart_with_items, "SAVE10")


def test_apply_coupon_sequential_with_monkeypatch(monkeypatch):
    cart = Cart()
    cart.add_item("Item", 100.0)

    monkeypatch.setattr('shopping.COUPONS', {'FIRST': 10})
    apply_coupon(cart, "FIRST")
    assert cart.total() == 90.0

    monkeypatch.setattr('shopping.COUPONS', {'SECOND': 20})

    new_cart = Cart()
    new_cart.add_item("Item", 100.0)
    apply_coupon(new_cart, "SECOND")
    assert new_cart.total() == 80.0

    with pytest.raises(ValueError, match="Invalid coupon"):
        apply_coupon(new_cart, "FIRST")


@pytest.mark.parametrize("coupon_code, expected_discount", [
    ("SAVE10", 10),
    ("HALF", 50),
])
def test_coupon_parametrized(cart_with_items, coupon_code, expected_discount):
    original_total = cart_with_items.total()
    apply_coupon(cart_with_items, coupon_code)
    assert cart_with_items.total() == original_total * (100 - expected_discount)
/ 100


def test_coupon_and_discount_chain(cart_with_items):
    cart_with_items.apply_discount(20)
    total_after_first = cart_with_items.total()
    apply_coupon(cart_with_items, "HALF")
    assert cart_with_items.total() == total_after_first * 0.5
```

Спецификация ввода

Ввод не требуется.

Спецификация вывода

После запуска программы запустятся тесты и покажется результат выполнения тестов.

Рисунки с результатами работы программы

```
src/test_cart.py::test_add_item PASSED                                                                    [  3%]
src/test_cart.py::test_negative_price PASSED                                                              [  6%]
src/test_cart.py::test_total PASSED                                                                       [ 10%]
src/test_cart.py::test_apply_discount_valid[0-expected_prices0] PASSED                                    [ 13%]
src/test_cart.py::test_apply_discount_valid[50-expected_prices1] PASSED                                   [ 16%]
src/test_cart.py::test_apply_discount_valid[100-expected_prices2] PASSED                                  [ 20%]
src/test_cart.py::test_apply_discount_invalid[-10] PASSED                                                 [ 23%]
src/test_cart.py::test_apply_discount_invalid[-1] PASSED                                                  [ 26%]
src/test_cart.py::test_apply_discount_invalid[101] PASSED                                                 [ 30%]
src/test_cart.py::test_apply_discount_invalid[150] PASSED                                                 [ 33%]
src/test_cart.py::test_empty_cart_total PASSED                                                            [ 36%]
src/test_cart.py::test_multiple_discounts PASSED                                                          [ 40%]
src/test_cart.py::test_add_item_after_discount PASSED                                                     [ 43%]
src/test_cart.py::test_log_purchase_success PASSED                                                        [ 46%]
src/test_cart.py::test_log_purchase_with_multiple_items PASSED                                            [ 50%]
src/test_cart.py::test_log_purchase_with_empty_item PASSED                                                [ 53%]
src/test_cart.py::test_log_purchase_with_http_error PASSED                                                [ 56%]
src/test_cart.py::test_log_purchase_integration_with_cart PASSED                                          [ 60%]
src/test_cart.py::test_apply_valid_coupon_save10 PASSED                                                   [ 63%]
src/test_cart.py::test_apply_valid_coupon_half PASSED                                                     [ 66%]
src/test_cart.py::test_apply_invalid_coupon PASSED                                                        [ 70%]
src/test_cart.py::test_apply_coupon_to_empty_cart PASSED                                                  [ 73%]
src/test_cart.py::test_apply_coupon_with_mocked_dict_patch PASSED                                         [ 76%]
src/test_cart.py::test_apply_coupon_with_multiple_mocked_coupons PASSED                                   [ 80%]
src/test_cart.py::test_apply_coupon_with_empty_mocked_dict PASSED                                         [ 83%]
src/test_cart.py::test_apply_coupon_with_monkeypatch PASSED                                               [ 86%]
src/test_cart.py::test_apply_coupon_sequential_with_monkeypatch PASSED                                    [ 90%]
src/test_cart.py::test_coupon_parametrized[SAVE10-10] PASSED                                              [ 93%]
src/test_cart.py::test_coupon_parametrized[HALF-50] PASSED                                                [ 96%]
src/test_cart.py::test_coupon_and_discount_chain PASSED                                                   [100%]

============================================================================ 30 passed in 0.61s ============================================================================
(.venv) PS E:\kurs3\SPP\LAB1\spp_po13\reports\Markovsky\6>
```

Задание 2:

Напишите тесты к реализованным функциям из лабораторной работы №1. Проверьте тривиальные и граничные случае, а также варианты, когда может возникнуть исключительная ситуация. Если при реализации не использовались отдельные функции, необходимо провести рефакторинг кода.

Код программы:

a1.py:

```
def calculate_range(numbers):
    if not numbers:
        raise ValueError("Последовательность пустая")

    min_val = min(numbers)
    max_val = max(numbers)
    return min_val, max_val, max_val - min_val


def parse_numbers_input(input_string):
    try:
        numbers = list(map(int, input_string.split()))
        return numbers
    except ValueError as e:
        raise ValueError("Последовательность некорректна - введите целые числа!")
from e


def main():
    try:
        input_str = input("Введите последовательность целых чисел: ")
        numbers = parse_numbers_input(input_str)
        min_val, max_val, range_val = calculate_range(numbers)
```

```
        print(f"Размах последовательности: {max_val} - {min_val} = {range_val}")
    except ValueError as e:
        print(e)


if __name__ == "__main__":
    main()
```

a2.py:

```
def find_longest_common_prefix(strings):
    if not strings:
        return ""

    strings.sort()
    min_word = strings[0]
    max_word = strings[-1]

    prefix = []
    for i, char in enumerate(min_word):
        if i < len(max_word) and char == max_word[i]:
            prefix.append(char)
        else:
            break

    return "".join(prefix)


def parse_strings_input(input_string):
    strings = list(map(str, input_string.split()))
    return strings


def main():
    try:
        input_str = input("Введите строки: ")
        strings = parse_strings_input(input_str)
        prefix = find_longest_common_prefix(strings)
        print(f"Самая длинная общая строка префикса среди списка строк:
{prefix}")
    except (ValueError, TypeError) as e:
        print(e)


if __name__ == "__main__":
    main()
```

test.py

```
import pytest
from a1 import calculate_range, parse_numbers_input
from a2 import find_longest_common_prefix, parse_strings_input


class TestCalculateRange:
    def test_normal_numbers(self):
        min_val, max_val, range_val = calculate_range([1, 2, 3, 4, 5])
        assert min_val == 1
        assert max_val == 5
        assert range_val == 4

    def test_negative_numbers(self):
        min_val, max_val, range_val = calculate_range([-10, -5, 0, 5, 10])
        assert min_val == -10
```

```
        assert max_val == 10
        assert range_val == 20

    def test_single_number(self):
        min_val, max_val, range_val = calculate_range([42])
        assert min_val == 42
        assert max_val == 42
        assert range_val == 0

    def test_all_same_numbers(self):
        min_val, max_val, range_val = calculate_range([5, 5, 5, 5])
        assert min_val == 5
        assert max_val == 5
        assert range_val == 0

    def test_empty_list(self):
        with pytest.raises(ValueError, match="Последовательность пустая"):
            calculate_range([])

    def test_large_numbers(self):
        min_val, max_val, range_val = calculate_range([-1000000, 0, 1000000])
        assert min_val == -1000000
        assert max_val == 1000000
        assert range_val == 2000000


class TestParseNumbersInput:
    def test_valid_input(self):
        numbers = parse_numbers_input("1 2 3 4 5")
        assert numbers == [1, 2, 3, 4, 5]

    def test_valid_input_with_negative(self):
        numbers = parse_numbers_input("-5 0 5 10")
        assert numbers == [-5, 0, 5, 10]

    def test_empty_input(self):
        numbers = parse_numbers_input("")
        assert numbers == []

    def test_single_number(self):
        numbers = parse_numbers_input("42")
        assert numbers == [42]

    def test_invalid_input_letters(self):
        with pytest.raises(ValueError,
                           match="Последовательность некорректна - введите целые
числа!"):
            parse_numbers_input("1 2 a b 5")

    def test_invalid_input_mixed(self):
        with pytest.raises(ValueError,
                           match="Последовательность некорректна - введите целые
числа!"):
            parse_numbers_input("1 2 3.5 4")


class TestFindLongestCommonPrefix:
    def test_normal_strings(self):
        result = find_longest_common_prefix(["flower", "flow", "flight"])
        assert result == "fl"

    def test_no_common_prefix(self):
        result = find_longest_common_prefix(["dog", "racecar", "car"])
        assert result == ""

    def test_all_identical(self):
        result = find_longest_common_prefix(["test", "test", "test"])
```

```
        assert result == "test"

    def test_one_string(self):
        result = find_longest_common_prefix(["hello"])
        assert result == "hello"

    def test_empty_list(self):
        result = find_longest_common_prefix([])
        assert result == ""

    def test_empty_strings(self):
        result = find_longest_common_prefix(["", "", ""])
        assert result == ""

    def test_different_lengths(self):
        result = find_longest_common_prefix(["python", "py", "pytest"])
        assert result == "py"

    def test_numbers_as_strings(self):
        result = find_longest_common_prefix(["123", "12", "1234"])
        assert result == "12"

    def test_case_sensitive(self):
        result = find_longest_common_prefix(["Hello", "hello", "HELLO"])
        assert result == ""

    def test_special_characters(self):
        result = find_longest_common_prefix(["!@#", "!@#123", "!@#abc"])
        assert result == "!@#"

    def test_long_prefix(self):
        result = find_longest_common_prefix(["abcdef", "abcde", "abcd"])
        assert result == "abcd"

    def test_prefix_only_in_some(self):
        result = find_longest_common_prefix(["prefix123", "prefix456", "other"])
        assert result == ""


class TestParseStringsInput:
    def test_valid_input(self):
        strings = parse_strings_input("hello world python")
        assert strings == ["hello", "world", "python"]

    def test_empty_input(self):
        strings = parse_strings_input("")
        assert strings == []

    def test_single_string(self):
        strings = parse_strings_input("hello")
        assert strings == ["hello"]

    def test_strings_with_numbers(self):
        strings = parse_strings_input("test123 test456")
        assert strings == ["test123", "test456"]

    def test_multiple_spaces(self):
        strings = parse_strings_input("hello    world   python")
        assert strings == ["hello", "world", "python"]


class TestIntegration:
    def test_calculate_range_integration(self):
        numbers = parse_numbers_input("10 20 30 40 50")
        min_val, max_val, range_val = calculate_range(numbers)
        assert min_val == 10
        assert max_val == 50
```

```
        assert range_val == 40

    def test_find_longest_common_prefix_integration(self):
        strings = parse_strings_input("flower flow flight")
        prefix = find_longest_common_prefix(strings)
        assert prefix == "fl"

    def test_calculate_range_with_negative_integration(self):
        numbers = parse_numbers_input("-100 -50 0 50")
        min_val, max_val, range_val = calculate_range(numbers)
        assert min_val == -100
        assert max_val == 50
        assert range_val == 150

    def test_empty_input_integration(self):
        numbers = parse_numbers_input("")
        with pytest.raises(ValueError, match="Последовательность пустая"):
            calculate_range(numbers)
```

Спецификация ввода

Ввод не требуется.

Спецификация вывода

После запуска программы запустятся тесты и покажется результат выполнения тестов.

Рисунки с результатами работы программы

```
✓ 33 tests passed   33 tests total, 0 ms
spp_po13/reports/Markovsky/6/src/2/test.py::TestFindLongestCommonPrefix::test_no_common_prefix PASSED [ 42%]
spp_po13/reports/Markovsky/6/src/2/test.py::TestFindLongestCommonPrefix::test_all_identical PASSED [ 45%]
spp_po13/reports/Markovsky/6/src/2/test.py::TestFindLongestCommonPrefix::test_one_string PASSED [ 48%]
spp_po13/reports/Markovsky/6/src/2/test.py::TestFindLongestCommonPrefix::test_empty_list PASSED [ 51%]
spp_po13/reports/Markovsky/6/src/2/test.py::TestFindLongestCommonPrefix::test_empty_strings PASSED [ 54%]
spp_po13/reports/Markovsky/6/src/2/test.py::TestFindLongestCommonPrefix::test_different_lengths PASSED [ 57%]
spp_po13/reports/Markovsky/6/src/2/test.py::TestFindLongestCommonPrefix::test_numbers_as_strings PASSED [ 60%]
spp_po13/reports/Markovsky/6/src/2/test.py::TestFindLongestCommonPrefix::test_case_sensitive PASSED [ 63%]
spp_po13/reports/Markovsky/6/src/2/test.py::TestFindLongestCommonPrefix::test_special_characters PASSED [ 66%]
spp_po13/reports/Markovsky/6/src/2/test.py::TestFindLongestCommonPrefix::test_long_prefix PASSED [ 69%]
spp_po13/reports/Markovsky/6/src/2/test.py::TestFindLongestCommonPrefix::test_prefix_only_in_some PASSED [ 72%]
spp_po13/reports/Markovsky/6/src/2/test.py::TestParseStringsInput::test_valid_input PASSED [ 75%]
spp_po13/reports/Markovsky/6/src/2/test.py::TestParseStringsInput::test_empty_input PASSED [ 78%]
spp_po13/reports/Markovsky/6/src/2/test.py::TestParseStringsInput::test_single_string PASSED [ 81%]
spp_po13/reports/Markovsky/6/src/2/test.py::TestParseStringsInput::test_strings_with_numbers PASSED [ 84%]
spp_po13/reports/Markovsky/6/src/2/test.py::TestParseStringsInput::test_multiple_spaces PASSED [ 87%]
spp_po13/reports/Markovsky/6/src/2/test.py::TestIntegration::test_calculate_range_integration PASSED [ 90%]
spp_po13/reports/Markovsky/6/src/2/test.py::TestIntegration::test_find_longest_common_prefix_integration PASSED [ 93%]
spp_po13/reports/Markovsky/6/src/2/test.py::TestIntegration::test_calculate_range_with_negative_integration PASSED [ 96%]
spp_po13/reports/Markovsky/6/src/2/test.py::TestIntegration::test_empty_input_integration PASSED [100%]

============================ 33 passed in 0.08s ============================

Process finished with exit code 0
```

Задание 3:

Написать тесты к методу, а затем реализовать сам метод по заданной спецификации. Реализуйте и потестируйте метод **int indexOfDifference(String str1, String str2)**, который сравнивает две строки и возвращает индекс той позиции, в которой они различаются. Например, **indexOfDifference("i am a machine" , "i am a robot")** должно вернуть 7.

<u>Спецификация метода:</u>

indexOfDifference (None , None) = TypeError
indexOfDifference ("", "") = -1
indexOfDifference ("", "abc ") = 0
indexOfDifference ("abc ", "") = 0

indexOfDifference ("abc ", " abc ") = -1
indexOfDifference ("ab", " abxyz ") = 2
indexOfDifference (" abcde ", " abxyz ") = 2
indexOfDifference (" abcde ", "xyz") = 0

Код программы:

string_diff.py:

```
def index_of_difference(str1, str2):
    if str1 is None or str2 is None:
        raise TypeError("Arguments cannot be None")

    str1 = str1.strip()
    str2 = str2.strip()
    min_length = min(len(str1), len(str2))

    for i in range(min_length):
        if str1[i] != str2[i]:
            return i

    if len(str1) != len(str2):
        return min_length

    return -1
```

test.py

```
# pylint: disable=too-many-public-methods
import pytest
from string_diff import index_of_difference


class TestIndexOfDifference:
    def test_both_none(self):
        with pytest.raises(TypeError, match="Arguments cannot be None"):
            index_of_difference(None, None)

    def test_first_none(self):
        with pytest.raises(TypeError, match="Arguments cannot be None"):
            index_of_difference(None, "abc")

    def test_second_none(self):
        with pytest.raises(TypeError, match="Arguments cannot be None"):
            index_of_difference("abc", None)

    def test_both_empty(self):
        result = index_of_difference("", "")
        assert result == -1

    def test_first_empty_second_not_empty(self):
        result = index_of_difference("", "abc")
        assert result == 0

    def test_second_empty_first_not_empty(self):
        result = index_of_difference("abc", "")
        assert result == 0

    def test_identical_strings(self):
        result = index_of_difference("abc", "abc")
        assert result == -1

    def test_identical_with_spaces(self):
        result = index_of_difference("abc ", " abc")
```

```
        assert result == -1

    def test_different_at_beginning(self):
        result = index_of_difference("abc", "xyz")
        assert result == 0

    def test_different_in_middle(self):
        result = index_of_difference("i am a machine", "i am a robot")
        assert result == 7

    def test_different_at_end(self):
        result = index_of_difference("abcde", "abcdf")
        assert result == 4

    def test_different_lengths_short_first(self):
        result = index_of_difference("ab", "abxyz")
        assert result == 2

    def test_different_lengths_short_second(self):
        result = index_of_difference("abcde", "abc")
        assert result == 3

    def test_different_lengths_with_diff_before_end(self):
        result = index_of_difference("abcd", "abxy")
        assert result == 2

    def test_specification_example_1(self):
        result = index_of_difference("abc ", " abc")
        assert result == -1

    def test_specification_example_2(self):
        result = index_of_difference("ab", " abxyz")
        assert result == 2

    def test_specification_example_3(self):
        result = index_of_difference(" abcde ", " abxyz ")
        assert result == 2

    def test_specification_example_4(self):
        result = index_of_difference(" abcde ", "xyz")
        assert result == 0

    def test_unicode_strings(self):
        result = index_of_difference("привет", "привет")
        assert result == -1

        result = index_of_difference("привет мир", "привет мир!")
        assert result == 10

    def test_numeric_strings(self):
        result = index_of_difference("12345", "12345")
        assert result == -1

        result = index_of_difference("12345", "12346")
        assert result == 4

    def test_case_sensitive(self):
        result = index_of_difference("Hello", "hello")
        assert result == 0

    def test_special_characters(self):
        result = index_of_difference("!@#$%", "!@#$%")
        assert result == -1

        result = index_of_difference("!@#$%", "!@#^%")
        assert result == 3
```

```
    def test_very_long_strings(self):
        long_str1 = "a" * 10000
        long_str2 = "a" * 9999 + "b"
        result = index_of_difference(long_str1, long_str2)
        assert result == 9999
```

Спецификация ввода

Ввод не требуется.

Спецификация вывода

После запуска программы запустятся тесты и покажется результат выполнения тестов.

Рисунки с результатами работы программы

```
✓ 23 tests passed  23 tests total, 0 ms

spp_po13/reports/Markovsky/6/src/3/test.py::TestIndexOfDifference::test_both_empty PASSED [ 17%]
spp_po13/reports/Markovsky/6/src/3/test.py::TestIndexOfDifference::test_first_empty_second_not_empty PASSED [ 21%]
spp_po13/reports/Markovsky/6/src/3/test.py::TestIndexOfDifference::test_second_empty_first_not_empty PASSED [ 26%]
spp_po13/reports/Markovsky/6/src/3/test.py::TestIndexOfDifference::test_identical_strings PASSED [ 30%]
spp_po13/reports/Markovsky/6/src/3/test.py::TestIndexOfDifference::test_identical_with_spaces PASSED [ 34%]
spp_po13/reports/Markovsky/6/src/3/test.py::TestIndexOfDifference::test_different_at_beginning PASSED [ 39%]
spp_po13/reports/Markovsky/6/src/3/test.py::TestIndexOfDifference::test_different_in_middle PASSED [ 43%]
spp_po13/reports/Markovsky/6/src/3/test.py::TestIndexOfDifference::test_different_at_end PASSED [ 47%]
spp_po13/reports/Markovsky/6/src/3/test.py::TestIndexOfDifference::test_different_lengths_short_first PASSED [ 52%]
spp_po13/reports/Markovsky/6/src/3/test.py::TestIndexOfDifference::test_different_lengths_short_second PASSED [ 56%]
spp_po13/reports/Markovsky/6/src/3/test.py::TestIndexOfDifference::test_different_lengths_with_diff_before_end PASSED [ 60%]
spp_po13/reports/Markovsky/6/src/3/test.py::TestIndexOfDifference::test_specification_example_1 PASSED [ 65%]
spp_po13/reports/Markovsky/6/src/3/test.py::TestIndexOfDifference::test_specification_example_2 PASSED [ 69%]
spp_po13/reports/Markovsky/6/src/3/test.py::TestIndexOfDifference::test_specification_example_3 PASSED [ 73%]
spp_po13/reports/Markovsky/6/src/3/test.py::TestIndexOfDifference::test_specification_example_4 PASSED [ 78%]
spp_po13/reports/Markovsky/6/src/3/test.py::TestIndexOfDifference::test_unicode_strings PASSED [ 82%]
spp_po13/reports/Markovsky/6/src/3/test.py::TestIndexOfDifference::test_numeric_strings PASSED [ 86%]
spp_po13/reports/Markovsky/6/src/3/test.py::TestIndexOfDifference::test_case_sensitive PASSED [ 91%]
spp_po13/reports/Markovsky/6/src/3/test.py::TestIndexOfDifference::test_special_characters PASSED [ 95%]
spp_po13/reports/Markovsky/6/src/3/test.py::TestIndexOfDifference::test_very_long_strings PASSED [100%]

============================ 23 passed in 0.05s ============================

Process finished with exit code 0
```

Вывод:

Я освоил приемы тестирования кода на примере использования пакета pytest.